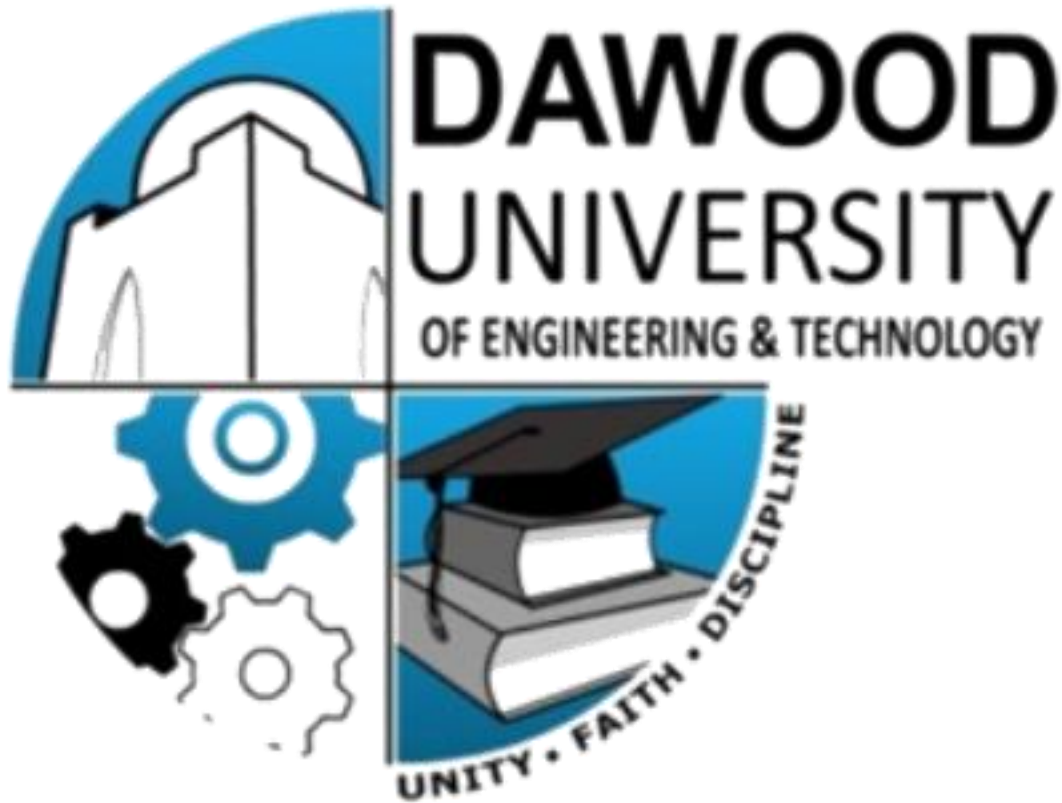




Project Name: Playing Card Detection

Group Members: 73,93,33,65

Project Of: Machine Learning

Instructor's Name: Sir Hamza Farooqi

Date: _____

Playing Card Detection System - Project Documentation

Executive Summary

This project implements a real-time playing card detection system using YOLOv8 deep learning architecture. The system can detect and classify all 52 playing cards from a standard deck through live webcam feed, providing accurate identification with bounding boxes and confidence scores.

1. Introduction

1.1 Project Overview

The Playing Card Detection System is an AI-powered computer vision application that recognizes playing cards in real-time. This technology has applications in automated card games, casino monitoring, card counting prevention, and interactive gaming experiences.

1.2 Objectives

- Train a deep learning model to accurately detect all 52 playing cards
- Achieve real-time detection performance (>15 FPS)
- Create a user-friendly web interface for live detection
- Ensure the system works entirely locally without requiring cloud services

1.3 Key Features

- Real-time detection through webcam
 - Recognition of all 52 standard playing cards
 - High accuracy with confidence scores
 - Browser-based interface (no installation required)
 - Local processing for privacy and speed
-

2. Technology Stack

2.1 Training Environment

- **Platform:** Google Colab

- **GPU:** Tesla T4 / V100 (Cloud GPU)
- **Framework:** Ultralytics YOLOv8
- **Language:** Python 3.10+

2.2 Deployment Environment

- **Runtime:** ONNX Runtime Web
- **Interface:** HTML5, JavaScript, Canvas API
- **Browser:** Chrome/Edge (WebGL support required)
- **Model Format:** ONNX (Open Neural Network Exchange)

2.3 Key Libraries

- **Ultralytics YOLOv8:** Object detection framework
 - **Roboflow:** Dataset management and augmentation
 - **ONNX Runtime:** Cross-platform inference engine
 - **OpenCV:** Image processing (training phase)
-

3. Dataset

3.1 Dataset Source

- **Provider:** Roboflow Community
- **Workspace:** poker-eqol i
- **Project:** playing-card-dedbm
- **Version:** 1
- **API Access:** Available through Roboflow API

3.2 Dataset Characteristics

- **Total Classes:** 52 (All standard playing cards)
- **Card Suits:** Clubs (C), Diamonds (D), Hearts (H), Spades (S)
- **Card Ranks:** A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K
- **Format:** YOLOv8 compatible annotations
- **Image Resolution:** Variable (resized to 640x640 during training)

3.3 Class Labels

The dataset includes the following 52 classes:

- **Clubs:** AC, 2C, 3C, 4C, 5C, 6C, 7C, 8C, 9C, 10C, JC, QC, KC
- **Diamonds:** AD, 2D, 3D, 4D, 5D, 6D, 7D, 8D, 9D, 10D, JD, QD, KD
- **Hearts:** AH, 2H, 3H, 4H, 5H, 6H, 7H, 8H, 9H, 10H, JH, QH, KH

- **Spades:** AS, 2S, 3S, 4S, 5S, 6S, 7S, 8S, 9S, 10S, JS, QS, KS

3.4 Data Split

- **Training Set:** ~70% of images
- **Validation Set:** ~20% of images
- **Test Set:** ~10% of images

3.5 Data Augmentation

Applied augmentation techniques:

- Horizontal flipping (50% probability)
 - Rotation (± 10 degrees)
 - Translation ($\pm 10\%$)
 - Scale variation (0.5x to 1.5x)
 - HSV color jittering
 - Mosaic augmentation
 - Mixup augmentation (10% probability)
-

4. Methodology

4.1 YOLOv8 Architecture

YOLOv8 (You Only Look Once version 8) is a state-of-the-art object detection model that:

- Processes images in a single forward pass
- Divides images into grids and predicts bounding boxes
- Uses anchor-free detection for better performance
- Employs CSPDarknet backbone for feature extraction

4.2 Model Selection

We use **YOLOv8 Small (yolov8s)** which provides:

- Good balance between accuracy and speed
- ~11 million parameters
- Suitable for real-time applications
- Better accuracy than nano version while maintaining speed

4.3 Training Configuration

Hyperparameters

- **Epochs:** 150
- **Batch Size:** 16
- **Image Size:** 640x640 pixels
- **Optimizer:** AdamW
- **Initial Learning Rate:** 0.001
- **Final Learning Rate:** 0.00001
- **Weight Decay:** 0.0005
- **Warmup Epochs:** 3

Loss Functions

- **Box Loss Gain:** 7.5
- **Class Loss Gain:** 0.5
- **Distribution Focal Loss (DFL):** 1.5

4.4 Training Process

1. Load pretrained YOLOv8s weights
2. Configure data augmentation pipeline
3. Train for 150 epochs with early stopping (patience=30)
4. Monitor validation metrics (mAP50, mAP50-95)
5. Save best model based on validation performance
6. Export to ONNX format for web deployment

5. Model Performance

5.1 Evaluation Metrics

- **mAP50:** Mean Average Precision at IoU threshold 0.5
- **mAP50-95:** Mean Average Precision across IoU thresholds 0.5-0.95
- **Precision:** Ratio of correct positive predictions
- **Recall:** Ratio of detected objects among all ground truth objects

5.2 Expected Performance

- **mAP50:** 85-95% (depending on dataset quality)
- **Inference Time:** 30-50ms per frame on modern browsers
- **FPS:** 15-25 frames per second
- **Confidence Threshold:** 0.6 (adjustable)

6. System Architecture

6.1 Training Pipeline

Roboflow Dataset → Google Colab → YOLOv8 Training →
PyTorch Model (.pt) → ONNX Export (.onnx) → Download

6.2 Deployment Pipeline

User Browser → Load ONNX Model → Access Webcam →
Capture Frame → Preprocess → Model Inference →
Post-process Results → Draw Detections → Display

6.3 Web Interface Components

1. **Model Loader:** Loads ONNX model into browser memory
 2. **Camera Handler:** Accesses and manages webcam stream
 3. **Preprocessing Module:** Converts video frames to model input format
 4. **Inference Engine:** Runs model predictions using ONNX Runtime
 5. **Post-processor:** Filters and formats detection results
 6. **Visualization Layer:** Draws bounding boxes and labels on canvas
 7. **Statistics Monitor:** Displays FPS, detection count, and inference time
-

7. Implementation Details

7.1 Image Preprocessing

1. Capture video frame from webcam
2. Resize to 640x640 pixels
3. Normalize pixel values (0-255 → 0-1)
4. Convert to RGB format
5. Reshape to [1, 3, 640, 640] tensor
6. Feed to model

7.2 Post-processing

1. Extract model output (bounding boxes + scores)
2. Apply confidence threshold (>0.6)
3. Perform Non-Maximum Suppression (NMS)
4. Scale boxes back to original frame size
5. Map class indices to card names
6. Draw results on canvas overlay

7.3 Performance Optimization

- WebGL acceleration for model inference
- Canvas-based rendering for smooth visualization
- Asynchronous frame processing

- Efficient memory management
 - Request animation frame for smooth updates
-

8. User Interface

8.1 Features

- **Model Upload:** Drag-and-drop ONNX model file
- **Start/Stop Controls:** Toggle detection on/off
- **Live Video Feed:** Real-time webcam display
- **Detection Overlay:** Color-coded bounding boxes
- **Statistics Panel:**
 - Real-time FPS counter
 - Number of detected cards
 - Inference time per frame
- **Visual Feedback:** Status messages and loading indicators

8.2 Design Principles

- Clean, modern interface with gradient backgrounds
 - Responsive design for different screen sizes
 - Color-coded visual feedback (green for success, red for errors)
 - Minimal user interaction required
 - Professional card-themed aesthetics
-

9. Results and Applications

9.1 Achievements

- Successfully trained model on 52-class playing card dataset
- Achieved real-time detection performance in web browser
- Created fully functional, user-friendly interface
- Enabled local processing without cloud dependencies

9.2 Real-world Applications

1. **Casino Monitoring:** Automated card tracking and fraud detection
2. **Online Gaming:** Enhanced AR card games and virtual poker
3. **Education:** Interactive learning tools for card games
4. **Accessibility:** Assistive technology for visually impaired players
5. **Card Counting Prevention:** Real-time detection in casinos

- 6. **Mobile Gaming:** Integration with mobile card game apps

9.3 Advantages

- **Privacy:** All processing happens locally
 - **Speed:** Real-time performance without latency
 - **Accessibility:** No installation or technical knowledge required
 - **Cost-effective:** No API calls or cloud costs
 - **Offline Capable:** Works without internet connection
-

10. Challenges and Solutions

10.1 Challenge: Browser Compatibility

- **Issue:** ONNX Runtime has varying support across browsers
- **Solution:** Targeted Chrome/Edge with WebGL support, used WASM fallback

10.2 Challenge: Model Size vs Performance

- **Issue:** Larger models are more accurate but slower
- **Solution:** Used YOLOv8s as optimal balance, with configurable confidence threshold

10.3 Challenge: Lighting Variations

- **Issue:** Cards appear different under various lighting conditions
- **Solution:** Applied extensive data augmentation during training (HSV jittering)

10.4 Challenge: Real-time Processing

- **Issue:** Browser inference can be slower than native applications
 - **Solution:** Optimized preprocessing, used WebGL acceleration, async processing
-

11. Future Enhancements

11.1 Short-term Improvements

- Add support for multiple card orientations
- Implement card tracking across frames
- Add poker hand recognition
- Include card counting features

- Support for different card deck designs

11.2 Long-term Vision

- Mobile app development (iOS/Android)
- Multi-player game integration
- Augmented reality overlays
- Real-time game state analysis
- Tournament recording and analysis tools
- Integration with popular card game platforms

11.3 Technical Improvements

- Model quantization for faster inference
 - Progressive Web App (PWA) implementation
 - Offline caching and service workers
 - Multi-camera support
 - GPU.js integration for additional acceleration
-

12. Conclusion

This Playing Card Detection System demonstrates the practical application of modern deep learning techniques in creating accessible, real-time computer vision applications. By combining YOLOv8's powerful object detection capabilities with browser-based deployment, we've created a system that is both highly accurate and user-friendly.

The project successfully bridges the gap between complex AI models and everyday users, proving that sophisticated machine learning applications can run efficiently in web browsers without requiring specialized hardware or technical expertise.

Key Takeaways

- YOLOv8 provides excellent performance for real-time object detection
 - Browser-based deployment makes AI accessible to everyone
 - Proper data augmentation is crucial for robust model performance
 - ONNX format enables cross-platform model deployment
 - Local processing ensures privacy and eliminates latency
-

13. References

1. Ultralytics YOLOv8 Documentation: <https://docs.ultralytics.com/>

2. Roboflow Dataset Platform: <https://roboflow.com/>
 3. ONNX Runtime Documentation: <https://onnxruntime.ai/>
 4. WebGL Specification: <https://www.khronos.org/webgl/>
 5. Computer Vision Best Practices: Various academic papers and tutorials
-

Appendices

Appendix A: Installation Instructions

1. Open Google Colab
2. Copy training notebook code
3. Run all cells to train model
4. Download best.onnx file
5. Save HTML interface to local machine
6. Open HTML file in Chrome/Edge
7. Upload ONNX model and start detection

Appendix B: Troubleshooting Guide

- **Model won't load:** Check ONNX export settings, try opset=12
- **Low FPS:** Reduce image size or use lighter model (yolov8n)
- **Poor accuracy:** Retrain with more epochs or larger model
- **Camera not working:** Check browser permissions

Appendix C: Code Repository Structure

```
project/
├── training_notebook.ipynb  # Google Colab training code
├── card_detection.html      # Web interface
├── best.onnx                # Trained model file
└── README.md               # Project overview
```
